TYPESAFE AI BLOG · COMPANY NEWS

Introducing System One Models & Jev

Diogo Almeida, founder, TypeSafe Sep 15, 2026

Source: https://typesafe.ai/blog/introducing-system-one-models-and-jev



Models have been superhuman at chat for years, so where is all the automation?

This has been my driving question for the last four years. At OpenAI, I helped build the methods that made language models useful at following instructions and talking with people. That work ended up as the research behind ChatGPT. At the time, I thought maybe chat models would lead to AGI, but despite the hype it became obvious to me that there was something really big missing.

After two years in stealth, countless technical challenges, and research breakthroughs... I am beyond excited to announce that today, TypeSafe AI is releasing our first **System One Model**: a new class of frontier models built to make fast, structured decisions that software can use directly.

We built a new stack entirely focused on automation: with a new model architecture, parallel sampler for maximum efficiency, and training method we call Reinforcement Learning for Calibrated Decisions (RLCD).

Our first public model is **Jev**, available today in early access. Jev achieves similar levels of intelligence on System One tasks compared to existing LLMs, while being two orders of magnitude faster and more efficient. While Jev gives up string generation, it's optimized for structured outputs and *can't* hallucinate.

Think of Jev as a frontier-intelligence function call: unstructured state in, typed probabilistic decisions out.

Extraordinary claims require extraordinary evidence so see below for the receipts. 💅

Frontiers, Old and New

	Existing LLMs	System One + Jev
Optimized with	Reinforcement Learning with Human Feedback (RLHF) / Reinforcement Learning with Verifiable Rewards (RLVR)	Reinforcement Learning for Calibrated Decisions (RLCD)
Optimizes for	Human preference: writeups and chat responses that human raters prefer. Verifiable rewards: outputs that can be programmatically verified.	Calibrated decisions: answers with epistemically honest probabilities on System One tasks.
Inputs	Unstructured data (e.g. text) with an emphasis on **sequential messages**.	Unstructured data (e.g. text) with an emphasis on **structured program state**.
Outputs	**Strings / generated text.** Strings are flexible and can be anything: chat responses, code, hallucinations, refusals, or even type-safe structured values. To be used by software, responses need to be parsed + validated. There is also always some risk that the AI goes off the rails.	**Type-safe structured values.** Possible outputs and structure are defined in advance. The model never makes type errors. All answers are accompanied with calibrated probabilities and confidence scores.
Sampling	**Sequential.** Generates one token at a time, each conditioned on the last.	**Parallel.** Generates all outputs in a single query. Incredibly efficient and hardware-aware.
Cost	Input tokens: from $0.20 to $10 / MTok. Output tokens: ~5x more expensive than input tokens.	Input tokens: $0.042 / MTok ($42 per billion tokens). Output tokens: FREE (too cheap to meter).
Speed	**End-to-end response time is** 3 to 329 seconds for frontier models. Fast enough for interfacing with humans, but a big bottleneck when integrated in code.	**End-to-end response time is 70ms-500ms** for TypeSafe. This can range from 40x-200x faster for the same levels of frontier intelligence for System One shaped queries.
Confidence	Even if prompted for a confidence estimate, models tend to be overconfident and inconsistent. If a model can do a task 95% of the time but doesn't say when it's in the 5%, it can't automate that task.	Always communicates confidence and uncertainty with every output. Calibrated: higher confidence means higher accuracy. More consistent: returns similar answers for similar inputs.

	Existing LLMs	System One + Jev
Use cases	**Human-in-the-loop tasks (chatbots, copilots, coding agents).** General and powerful, but requires human oversight because their freedom also means they might go off the rails. **Verifiable problems (math proofs, kernel optimization).** When correctness can be checked cheaply and automatically, LLMs can generate, test, and iterate until they find something that works. **Demos.** The flexibility of strings allows it to be incredible for quickly making prototypes that only work sometimes.	**AI-Powered Workflows / smart if-statements.** Structured outputs slot into ordinary software as fuzzy decision rules: classify, route, score, extract, or branch where hand-written logic is too brittle. The surrounding code constrains their freedom, making them easier to compose into reliable systems. **Map-reducing over big data.** Turn petabytes of data into features and insights. **Real-time applications.** 100ms speeds means you can use AI in your applications where UX is critical. **Verify everything.** Score, judge, verify, guardrail, and detect jailbreaks of LLM prompts, reasoning traces, and/or outputs.

Evidence / Technical Results

We love skeptics, and are skeptics ourselves.

There are some claims you can easily verify:

- **Speed per call:** We truly are that fast, though our published evals are generally run from our laptops on the West Coast (this is where our service is currently based).
- **Cost per call:** We make our pricing transparent. We can't prove it isn't subsidized; we'll need the long-term to prove the sustainability of our pricing (which we expect to go down, not up).
- **No type errors**: This would be an easy thing to falsify with just a single counter-example, but it is mathematically impossible.

For our bolder claims, we want to provide as much nuance as we can.

Side-by-side demonstration

Our side-by-side demo shows a key difference between our models and LLMs: Jev outputs all probabilities in parallel instead of autoregressively generating by token. Strings are extremely powerful and general, but costly. "Giving up" strings actually gives us a lot of superpowers!



Video (0:15) — still frame shown; play it in the original post.

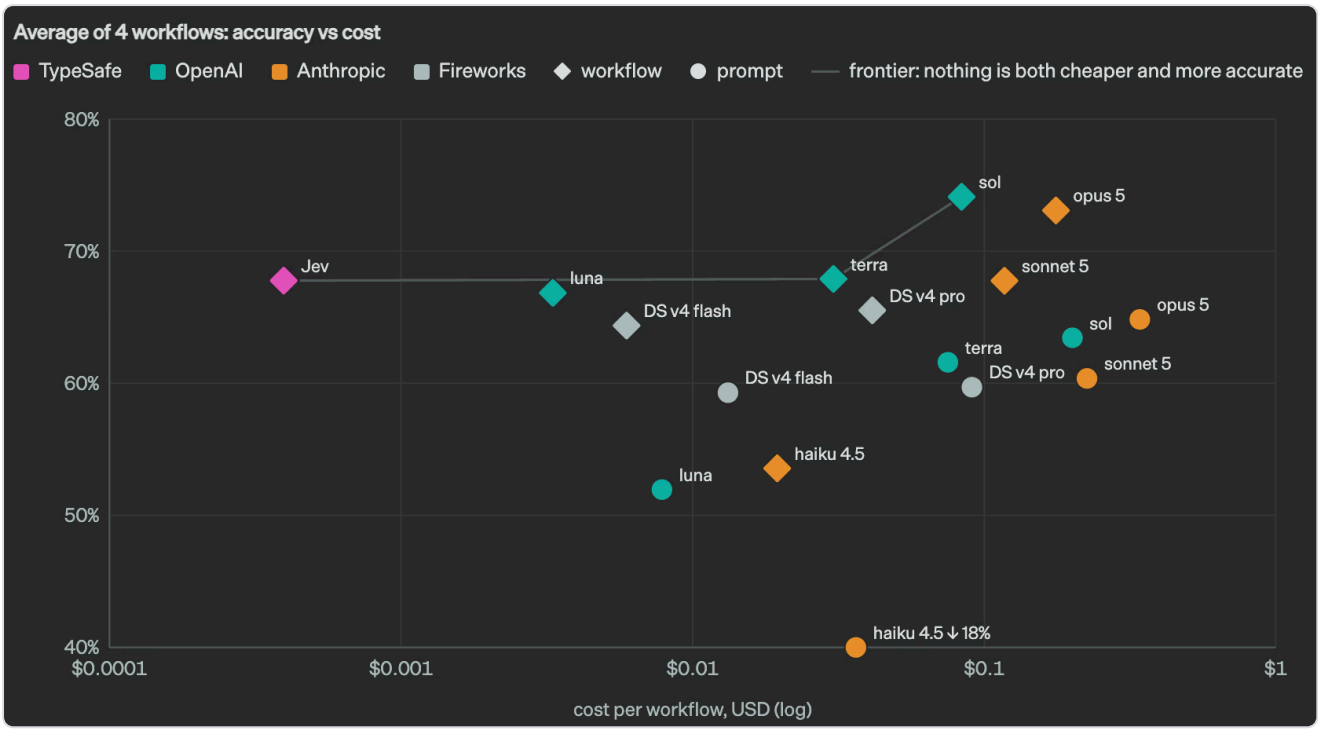
NUANCE

- For people with early access to TypeSafe, here is the actual query.
  - The query is highly simplified and `questions` were chosen to have descriptive, human-readable keys so that the output on the screen is understandable.
  - The `state` is also a short, dense, and detailed paragraph, to emphasize the difference in sampling methodology. The relatively shorter input paints our model in an advantageous light.
- For the keen eyed, for the recorded run, the only disagreement with GPT-5.6 Terra is on "Churn likelihood level". The actual answer seems genuinely ambiguous to us.
- We used GPT-5.6 Terra with default reasoning for this example, because we've found it to be the most comparable at intelligence to Jev on average.
- Fun fact: a similar demo was what convinced us to go all-in in the direction of System One Models!

Workflow evals

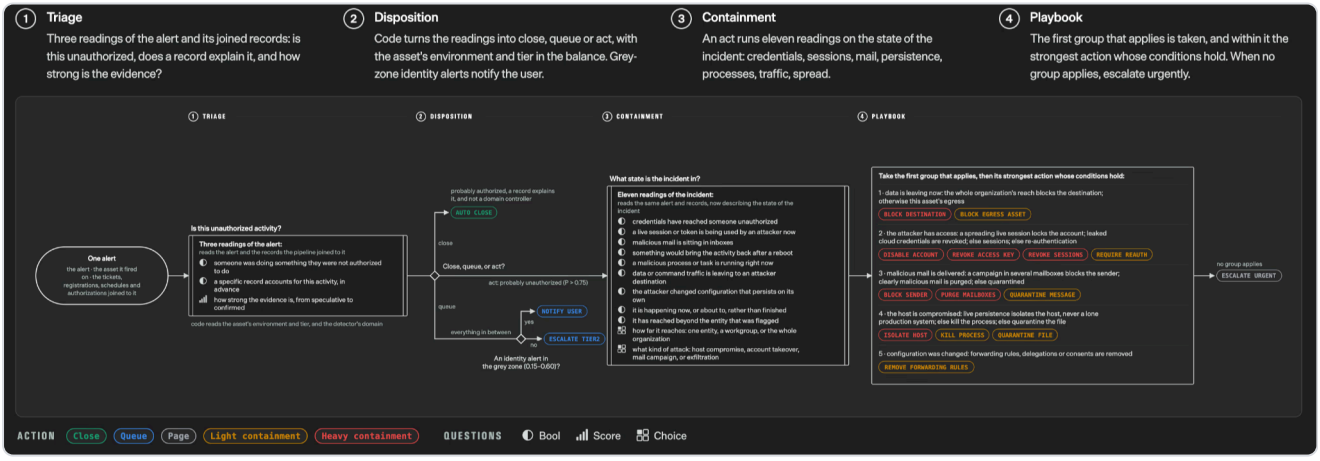
We made a new type of evaluation to measure how well AI works within code. We don't optimize for a ground truth classification or allow the harness and model to change (potentially allowing for overfitting via harness engineering). Instead, we assume there is a correct compute graph (a "workflow" represented in code) and use the predictions of the largest, smartest, and most expensive external models as reference probabilities.

Rephrased: every model gets the same workflow. We test how they compare to the average of the smartest models (in this case, Astra and Fable).



Jev is off the charts – owning the Pareto frontier for almost 2 orders of magnitude. We also compare to models with a generated prompt doing all the logic in their chain-of-thought, but this tends to do significantly worse than using the workflow itself.

Note that the calls here are significantly more complex than the side-by-side demonstration above. That's because they're more representative of the types of production workloads needed for true business automation. Below is the simplest of the 4 workflows we're publishing:



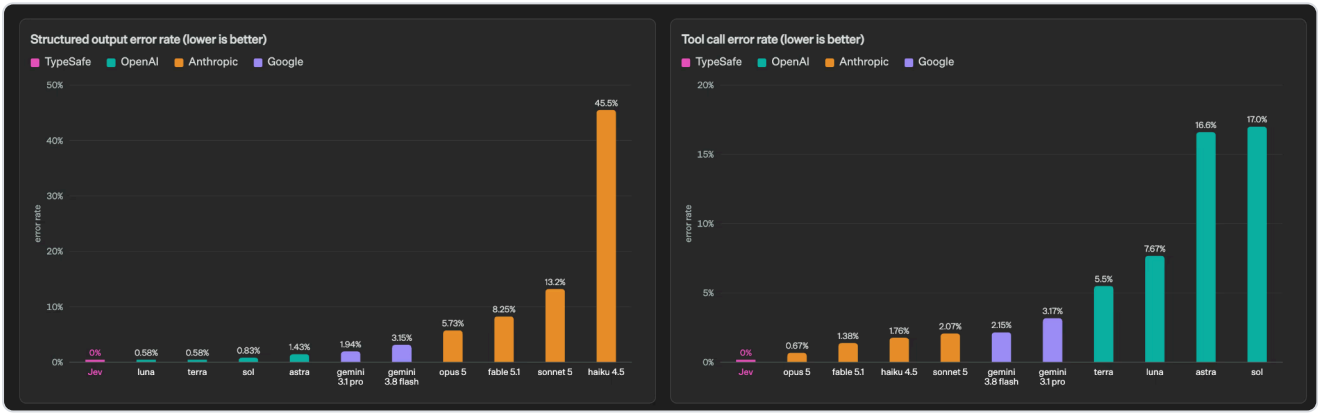
The most reliable real-world workflows tend to have many independent, decomposed questions, with fine-grained behavior that's dependent on probabilities instead of discrete decisions. The end result is discrete branching, but how we get to a final answer involves a lot of domain-specific engineering that needs to be done highly consistently.

See [our workflow evals site] for all the details: examples, disagreements, full queries, and each workflow.

NUANCE

- This is where the claims of 193.6x faster, 444.6x cheaper on our home page comes from, and we expect that these are on the higher end of real world gains.
- These content of these workflows were not deliberately chosen nor constructed to make our model look good, and are not in our training distribution. However, they were made by individuals on our model capabilities team, so some bias could exist.
- We use the average of GPT-6 Astra and Fable 5.1 as the reference answer, which biases answers towards OpenAI and Anthropic's models. We likely underestimate the relative performance of our model and DeepSeek's models.
- The LLMs use our System One LLM wrapper, which constrains LLMs to output structured decisions compatible with our API. We have found this to be the most accurate way to get decisions from LLMs, but this tends to be slower and more expensive than giving decisions without probabilities.

Hallucination and Type-safety



Hallucination and type-safety are intrinsically related, and we think the latter is table stakes for automation. Having a hallucinated tool call is inconvenient in an agent, but is an absolute deal-breaker if it's part of a system with latency guarantees or it's buried several layers deep in a dependency chain. Existing models, *no matter how smart*, still hallucinate and have type errors.

NUANCE

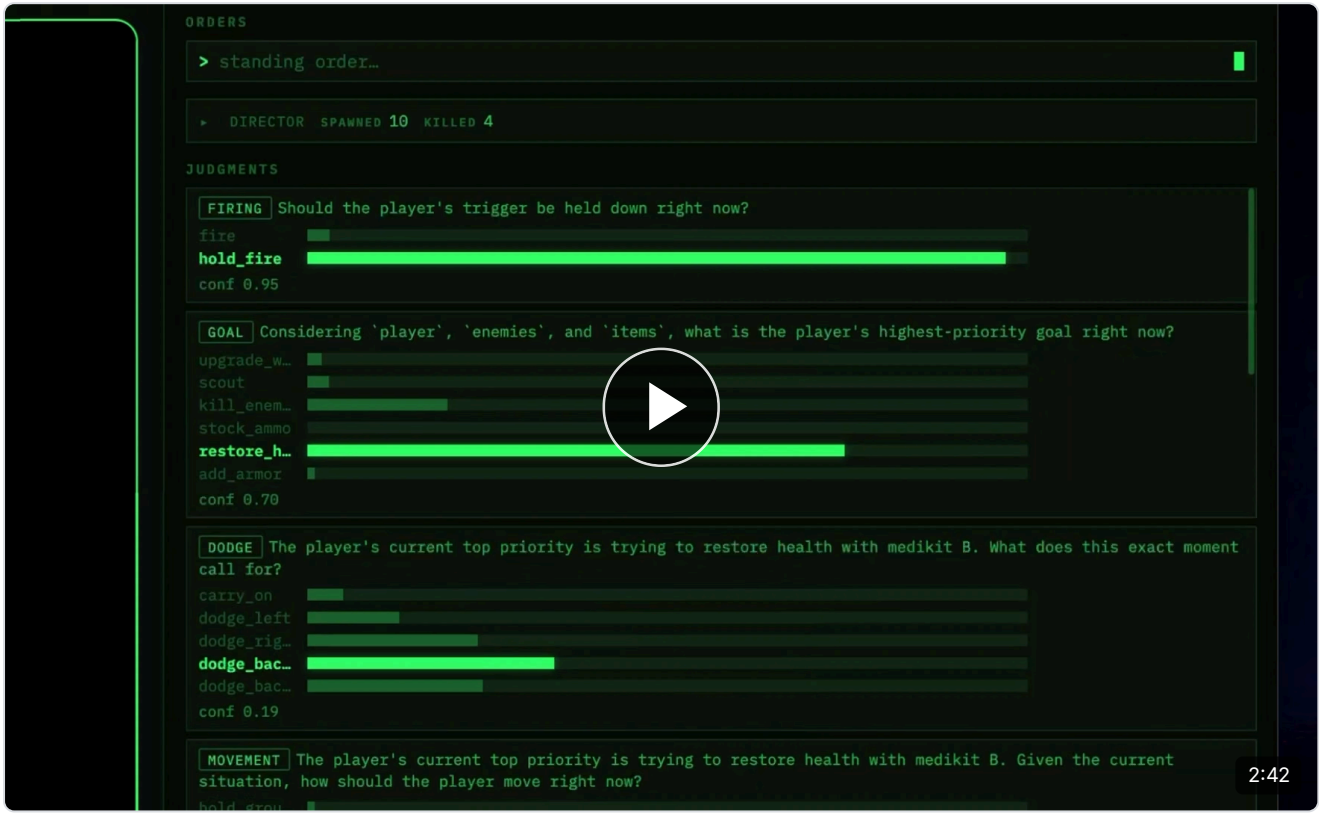
- The numbers for LLMs are from OpenRouter i.e., there almost certainly is bias here: more complex queries might be routed to better models.
- Our number is not empirical. Schema matching is guaranteed, thus we can confidently add 0% into the plots.

Fun Demos

Perhaps the most exciting part of our work is enabling new use cases. We have a lot more to show you, but here are a couple of the team's favorites:

Doom

We love how this doomo doomonstrates real-time intelligence and what can be doone with code + AI. The engineer behind it was worried about making 10 queries a second (which ends up costing ~$7/hour), but the rest of us agreed that was lower than expected! This is so fun we intend to not only release an in-depth walkthrough, but also host some events to hack on this.

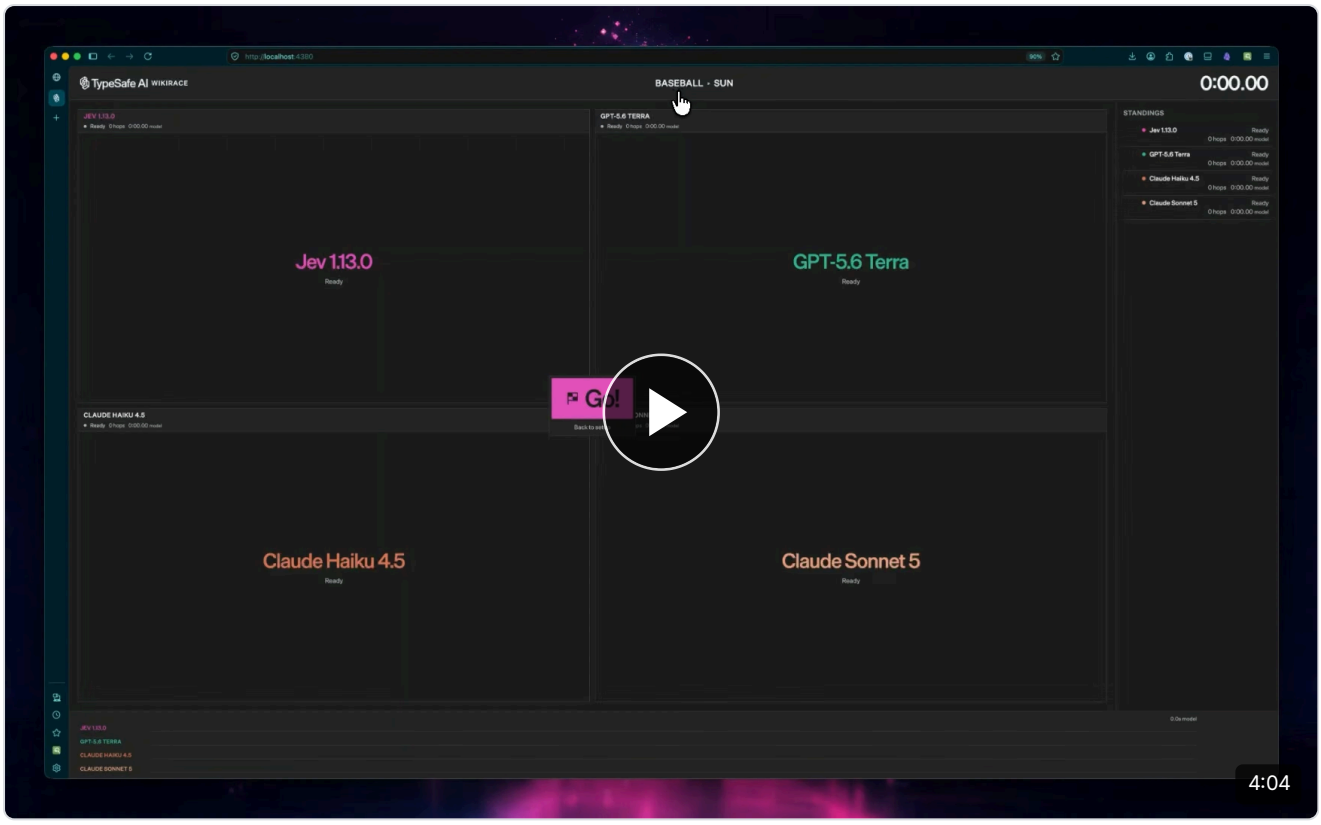


Video (2:42) — still frame shown; play it in the original post.

> **NUANCE**
>
> - The demo is on structured state as a data structure with text, not on images (yet…)
> - A non-AI doom bot could play better, but we wanted a bot that was reactive to different representations of game state, and most importantly… following instructions was cool as heck!

Wikiracing

The objective of the game is to start on one Wikipedia page and reach a specific other Wikipedia page using only links you come across while traversing. Each step can mean choosing between hundreds to thousands of links! It's a great playground for demonstrating not just intelligence-per-second, but also the compounding benefits of not hallucinating with high-cardinality choices.



Video (4:04) — still frame shown; play it in the original post.

NUANCE

- As far as we know, it was completely random that both the 2nd and 3rd challenges started with “Rubber Duck.” The author only noticed when the team pointed it out.
- Our speedups here tend to be a lot less than in previous demos. That’s because this is against the non-reasoning modes of the models (except Astra which was set to the lowest reasoning setting). This is also why Jev tended to finish in fewer steps (a sign of greater intelligence). This was to make the demo more bearable to watch. The LLMs look much worse at this task than with reasoning enabled.
- Jev supports a cardinality up to 255. For the higher cardinality choices, we do a 2 stage-system of scoring independently then making an explicit choice, hence the occassional slowdown.

What’s next

We’re still in Jev’s early days. We have a lot more in the pipeline and are so excited to keep on shipping 🔥.

Today, we are opening early access and bringing developers off the waitlist as quickly as we can. We want to hear which decisions you need to automate, where Jev works, and where it falls short. Tell us what sci-fi you want to build!!

We started TypeSafe because we believe that AI needs an interface software could depend on. We can't wait to see new use cases *continuously diffuse* through the community and economy.

We Give A FAQ

Where do the names "System One Models" and "Jev" come from?

We were inspired by Daniel Kahneman, Thinking, Fast and Slow. The model class name draws on the distinction between fast, intuitive System 1 thinking and slow, deliberate System 2 reasoning.

"System 1 thinking" has also implied error-prone. For reasons we will get into in the future, we believe System One Models can be made more reliable than its alternatives.

We named Jev after William Stanley Jevons. We expect machine intelligence to follow a similar path to coal, after steam-engine efficiency led to an increase in demand. Every order of magnitude drop in the cost of intelligence unlocks orders of magnitude more use cases.

Why was a new training algorithm needed?

Every lab optimizes for the same task during Reinforcement Learning with Human Feedback (RLHF): *produce the text that a human rater prefers*. That was the right task for a chat product, but it is the wrong task for automation. This is what we call the bitterest lesson: optimizing for the right task matters more than data, compute, or algorithms.

RLVR is great for tasks with simple programmatic verification, but most real-world judgement tasks don't fit into that shape. This tends to cause spikey / non-robust intelligence.

What use cases is Jev good for?

We've found diverse use cases for Jev across industries. We outline some in our docs, and are excited to see what else developers build.

Is Jev just a smaller LLM?

Jev is neither small nor an LLM, hence being off the intelligence Pareto curve.

How does Jev perform against public benchmarks?

We deliberately chose *not* to publish performance against *public* benchmarks. In fact, we plan to only have one-off evals when we make product updates.

Given that we are opening up a new frontier for models, we're pushing for more useful best practices:

- Put no weight on public benchmarks.
- Encourage users to create their own evals for their use cases (System One tasks are much easier to evaluate).
- Disclose the nuance in your evals.
- De-emphasizing benchmarks **even when you're ahead.**

See this blog post about our philosophy around optimizing for benchmarks.

Where does our training data come from?

TypeSafe is primarily a data research lab, which is how the biggest results in AI get made. We make all the data ourselves. We wouldn't train on your data even if you asked us to (no offense). We do some pretty sophisticated stuff, but if you want to find out more, we'd have to hire you.

These results are kinda crazy - how is it possible?

See our blog post on AI's bitterest lesson. The short answer is that you get what you optimize for. LLMs optimized for being incredible chatbots and copilots, which made them superhuman at those humans-in-the-loop tasks. We're optimizing for the System One interface.